

目录

一、 CUDA 模型	1
(一) CUDA 硬件模型	1
(二) CUDA 存储模型图	2
(三) NVIDIA GPU 架构演进	4
(四) CUDA 编程模型	6
1. CUDA 核函数	6
2. 其它常用函数	7
3. 编程约束	8
4. CUDA 程序基本流程	9
5. 主要优化方法	10
二、 实验环境	11
(一) 实验环境简介	11
(二) 环境配置流程	12
三、 翻译	13
四、 实验一: SAXPY	13
(一) 实验背景	13
(二) 实验代码浅析	13
1. 主程序	13
2. 算法实现	13
(三) 实验过程	14
(四) 结果与结论	14

五、 实验二：并行前缀和	16
(一) 实验背景.....	16
(二) 实验代码浅析.....	16
(三) 实验过程.....	17
1. 算法设计	17
2. 实现过程	18
(四) 结果与结论	19
六、 实验三：简单圆形渲染器	19
(一) 实验代码浅析.....	20
1. 代码概述	20
2. 渲染过程浅析	20
(二) 实验过程与结果	21
参考文献.....	25

一、CUDA 模型

CUDA (Compute Unified Device Architecture) 是由 NVIDIA 开发的并行计算平台和编程模型，使开发人员能够充分利用 GPU 的强大并行计算能力。

(一) CUDA 硬件模型

CUDA 硬件模型主要由以下几个关键组件组成：GPU 处理集群（GPU Processing Clusters, GPC）、纹理处理集群（Texture Processing Clusters, TPC）、流式多处理器（Streaming Multiprocessors, SM）和 CUDA 核心（Streaming Processor, SP）。

GPC 是 CUDA GPU 架构中最高级别的结构单元。每个 GPC 包含若干 TPC 和其他一些共享资源，例如 L2 缓存。GPC 负责处理大量并行任务，实现高吞吐量的计算能力。一个 GPU 通常包含多个 GPC，从而可以并行处理更多的任务。每个 GPC 内部分布着多个 TPC，TPC 是次一级的集群，主要负责处理纹理相关的计算任务。TPC 还包含若干 SM，它是 CUDA 硬件架构中的基本计算单元。图1是来自 NVIDIA Ampere 架构白皮书^[1]中的 GPU 核心结构图，由于我使用的 GPU 是 Ampere 架构的，因此我选用了同样的架构。



图 1 GA100 核心全景结构图

如图2所示，每个流式多处理器包含若干 CUDA 核心、调度器 (scheduler)、寄存器文件、共享内存和其他辅助硬件。SM 负责执行具体的 CUDA 线程块 (thread blocks)。CUDA 核心是最基本的计算单元，执行 CUDA 程序中的线程指令。每个 SM 中的 CUDA 核心可以执行浮点运算 (FP32、FP64) 和整数运算 (INT32)，以及深度学习加速运算 (Tensor Cores)。每个 SM 包含的 CUDA 核心数量通常较多，这使得 GPU 能够高效地执行并行计算任务。需要注意的是，当我们提到一个 GPU 的 CUDA 核心总

数时，这通常指的是 FP32 核心的总数，因为 FP32 运算是 GPU 中最常见的运算类型。



图 2 GA100 核心 SM 结构图

在硬件层级关系中，GPC 包含多个 TPC，TPC 包含多个 SM，而 SM 包含多个 CUDA 核心。这个层级结构确保了 CUDA GPU 能够高效地处理大规模并行计算任务。例如，假设一个 CUDA GPU 具有 4 个 GPC，每个 GPC 包含 2 个 TPC，每个 TPC 包含 4 个 SM，而每个 SM 包含 128 个 CUDA 核心。这意味着该 GPU 总共有 4096 个 CUDA 核心。这种架构使得 GPU 能够高效地执行并行计算任务，并在各种应用场景中提供卓越的性能。

(二) CUDA 存储模型图

CUDA 的存储架构是理解 CUDA 编程的关键，它决定了数据如何在主机 (Host) 和设备 (Device) 之间传输，以及如何在设备内存之间访问和共享。CUDA 存储架构包

括多种类型的内存，每种内存都有其特定的用途和访问机制。这些内存类型包括全局内存、常量内存、纹理内存、共享内存、寄存器和局部内存。理解这些存储类型与 CUDA 硬件结构的对应关系，可以帮助更好地优化 CUDA 程序。

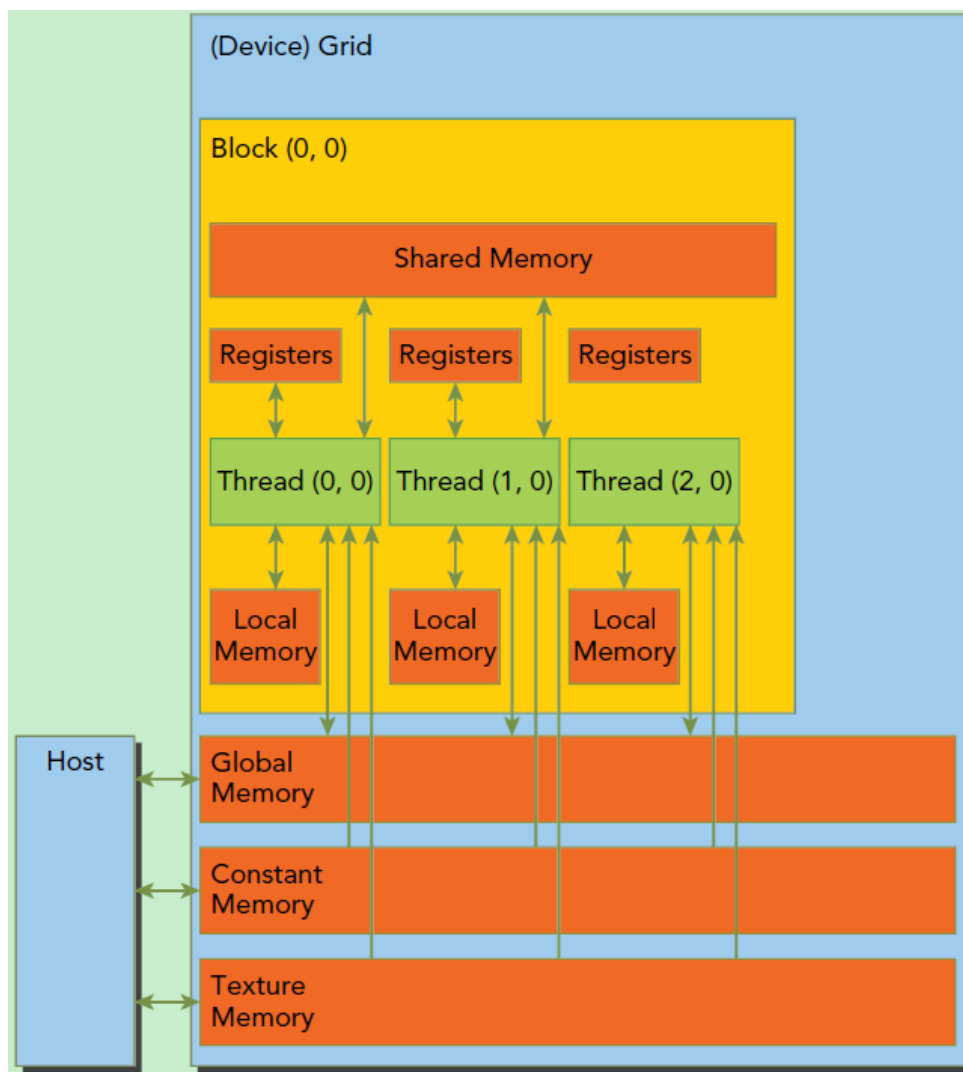


图 3 CUDA 存储模型

全局内存是设备内存中容量最大的一部分，可以由所有线程访问。全局内存用于存储需要在多个线程块之间共享的数据。尽管其访问速度较慢，但由于其容量大，适合存储大量数据。在硬件结构中，全局内存位于 GPU 的所有流式多处理器 (SM) 之外，是一个共享的资源。全局内存的数据传输需要通过 `cudaMemcpy` 函数在主机和设备之间进行。图3中，最底层的全局内存是连接主机和设备的重要桥梁。

常量内存是只读内存，专门用于存储在整个核函数执行过程中不会改变的值。其容量较小，但在所有线程访问相同位置时具有较低的访问延迟。常量内存的内容在核函数执行之前通过 `cudaMemcpyToSymbol` 从主机复制到设备。在硬件架构中，常量内存和全局内存一样，位于所有 SM 之外，由一个专门的单元进行管理。图3中展示了常量内

存位于全局内存旁边，但其访问速度和模式有所不同。

纹理内存也是一种只读内存，主要用于图像处理和其他需要高效随机访问的数据场景。纹理内存通过专门的纹理单元访问，可以显著提高 2D 和 3D 图像数据的处理效率。在硬件结构中，纹理内存由 TPC 中的纹理单元管理。图3中，纹理内存紧邻全局内存和常量内存，表示其在设备内存架构中的位置。

共享内存是每个线程块内的所有线程共享的高速内存。共享内存具有低访问延迟，适用于需要频繁访问和修改的数据。每个线程块在执行核函数时会分配一定大小的共享内存，线程块内的所有线程可以高效地访问和通信。在硬件结构中，共享内存位于每个 SM 内部，由 SM 管理。图3中，共享内存位于线程块内部。

寄存器是每个线程专用的高速存储器，用于存储线程执行过程中需要频繁访问的变量。寄存器具有最快的访问速度，但数量有限。每个线程在执行时会分配一定数量的寄存器，用于存储局部变量。在硬件结构中，寄存器也是 SM 内部的一部分，每个 SM 管理其内部的所有寄存器。图3中，寄存器紧邻线程，显示了其与每个线程的紧密关系。

局部内存是每个线程的私有内存，用于存储线程的私有数据，如自动变量和超出寄存器数量限制的变量。尽管名为局部内存，它实际上是设备全局内存的一部分，因此访问速度较慢。在硬件结构中，局部内存与全局内存相同，由设备内存管理单元管理。图3中，局部内存位于每个线程下方，表示其在线程执行中的位置。

图3展示了 CUDA 存储架构的各个层次，显示了不同类型的内存以及它们之间的关系。主机通过全局内存、常量内存和纹理内存与设备进行数据交互，而在设备内部，线程通过寄存器、局部内存和共享内存高效地处理数据。这种多层次的存储架构使得 CUDA 能够高效地执行大规模并行计算任务，充分利用 GPU 的计算能力。

（三）NVIDIA GPU 架构演进

NVIDIA GPU 架构经历了多次演进，每一代架构都在性能、效率和功能上有显著提升。这些架构演进不仅增强了 GPU 的计算能力，还引入了许多新技术和新特性，使其在高性能计算和人工智能领域发挥了重要作用。

Fermi 架构标志着 NVIDIA GPU 向通用计算平台的转变。Fermi 引入了流式多处理器的概念，每个 SM 包含 32 个 CUDA 核心。这一架构支持 ECC 内存和 C++ 编程，大幅提升了计算精度和编程便利性。此外，Fermi 还引入了 L1 和 L2 缓存体系结构，提高了内存访问效率。

Kepler 架构在 Fermi 的基础上进一步优化，主要特点是提高了能效比。每个 Kepler SMX（增强型 SM）包含 192 个 CUDA 核心。Kepler 引入了动态并行和 Hyper-Q 技术，前者允许 GPU 内核函数调用其他内核函数，后者通过多重队列机制提高了多线程执行

效率。这些改进使得 Kepler 架构在高性能计算和图形处理方面表现出色。

Maxwell 架构注重优化电源效率和性能,每个 SM 包含 128 个 CUDA 核心。Maxwell 引入了新的内存压缩技术,显著降低了内存带宽需求。这一架构的设计目标是提高每瓦特的性能,使得 GPU 在高性能和低功耗之间取得了良好的平衡。

Pascal 架构带来了深度学习加速的新突破。Pascal 引入了支持混合精度计算的 Tensor Cores,显著提升了深度学习训练和推理的性能。Pascal 还支持 NVLink 高速互联技术,提供了更高的带宽以实现多 GPU 之间的快速数据传输。这些特性使 Pascal 在 AI 和高性能计算领域取得了巨大的成功。

Volta 架构进一步增强了 Tensor Cores 的性能,每个 SM 包含 64 个 CUDA 核心和 8 个 Tensor Cores。Volta 引入了新的可编程内存访问模式,极大地提高了计算效率。Volta 架构在 AI 计算方面表现尤为突出,广泛应用于深度学习和科学计算领域。

Turing 架构引入了 RT Cores 用于实时光线追踪和 Tensor Cores 用于 AI 加速,每个 SM 包含 64 个 CUDA 核心、8 个 Tensor Cores 和 2 个 RT Cores。Turing 架构大幅提升了图形渲染和 AI 计算的性能,开启了实时光线追踪技术的新时代,使得游戏和图形应用能够呈现更加逼真的画面效果。

Ampere 架构进一步增强了 Tensor Cores 和 RT Cores 的性能。每个 SM 包含 128 个 CUDA 核心、4 个 Tensor Cores 和 1 个 RT Core。Ampere 架构支持第三代 Tensor Cores,可以进行更高效的 AI 计算,新的 RT Cores 提供更快光线追踪性能。此外,Ampere 架构还引入了多实例 GPU (MIG) 技术,允许将一个 GPU 分割成多个独立的实例,以便更高效地利用 GPU 资源。Ampere 架构在深度学习、科学计算和图形处理等多个领域表现出色。

Hopper 架构(以 GH100 为例)相较于前代 Amper 架构(GA100),在多个方面进行了提升。首先,Hopper 架构中的流式多处理器数量从 128 增加到 144,每个 SM 中的 FP32 CUDA 核心数量也翻倍,从 64 个增加到 128 个。此外,Hopper 引入了第四代 Tensor Cores,支持更高效的深度学习计算。同时,Hopper 架构支持最新的 HBM3 和 HBM2e 显存,带宽从 1555GB/s 提升至 3352GB/s,极大提高了内存访问速度。L2 缓存的容量也从 40MB 增加到 50MB,进一步提升了数据存储和访问效率。

在硬件创新方面,Hopper 架构引入了 Tensor Memory Accelerator (TMA)。TMA 是一种专用硬件单元,用于优化全局内存和共享内存之间的数据传输。与之前需要使用特殊指令进行异步内存复制不同,TMA 能够在不增加线程负担的情况下高效执行异步拷贝操作。单个线程可以启动 TMA 操作,创建一个复制描述符,之后的地址生成和数据移动由 TMA 硬件处理。TMA 不仅支持线性拷贝,还支持最多五维的张量操作,处

理地址、步幅、边界检查等复杂任务，进一步提升了数据搬运的效率和灵活性。

Hopper 架构还引入了一个新的层次结构——Thread Block Cluster。Thread Block Cluster 可以看作是“Block of Blocks”，为编程提供了新的视角和更大的灵活性。Thread Block Cluster 中的所有 Block 被调度到同一个 GPC 上，享有更快的本地同步和内存共享。在这种架构下，同一个 Cluster 中的 Block 可以直接读写其他 Block 的共享内存，形成了 Cluster Distributed Shared Memory (DSMEM)，这极大地优化了跨 Block 的数据交换和通信效率。

异步屏障 (Asynchronous Barriers) 是 Hopper 架构中的另一项重要特性。与传统的同步机制不同，异步屏障将同步过程分为两个步骤：首先，线程完成各自的任务后标记为“到达” (Arrive)，这一操作是非阻塞的，可以自由执行其他任务；然后，所有线程执行完独立任务后触发“等待” (Wait)，这一操作是阻塞的，直到所有线程都到达后才继续执行。这种机制提高了线程并行性，减少了等待时间。Hopper 架构还引入了 Asynchronous Transaction Barrier，用于实现单向内存复制和消息传递，进一步提升了数据移动效率。

(四) CUDA 编程模型

1. CUDA 核函数

CUDA 编程模型使开发人员能够利用 GPU 的并行计算能力，通过定义和调用核函数 (kernel function) 来执行大规模并行计算任务。核函数是 CUDA 程序的核心组件，定义了 GPU 上执行的代码，并通过配置网格和线程块的维度来管理并行计算的规模和粒度。

核函数使用 `__global__` 修饰符定义，它在设备上执行，由主机调用。核函数的主要作用是在 GPU 上并行处理数据，每个线程执行相同的代码，但操作不同的数据元素。以下是一个简单的矩阵加法核函数示例：

```
1 __global__ void matrixAdd(float* A, float* B, float* C, int N) {  
2     int i = blockIdx.x * blockDim.x + threadIdx.x;  
3     if (i < N) {  
4         C[i] = A[i] + B[i];  
5     }  
6 }
```

核函数示例

在这个核函数中，我们使用了 CUDA 提供的几个内置变量来确定每个线程在网格和线程块中的位置。blockIdx 表示线程块在网格中的索引，blockDim 表示每个线程块

中的线程数量，`threadIdx` 表示线程在线程块中的索引。这些内置变量帮助计算每个线程处理的数据元素的位置。此外，CUDA 还提供了 `gridDim` 变量，用于表示网格中线程块的数量。

这个示例程序中所使用的线程块和网格都是一维的，`blockIdx.x` 是当前线程块在一维网格中的索引，`blockDim.x` 是每个线程块中线程的数量，`threadIdx.x` 是当前线程在线程块中的索引。通过这三个变量的组合，可以唯一确定每个线程在整个网格中的全局索引 `i`。这个全局索引 `i` 对应于一维数据结构中的位置。在 CUDA 编程中，网格和线程块可以是一维、二维或者三维的。使用一维的网格和线程块是最简单和常见的情况，适用于线性数据结构（如数组、向量）或简单的计算任务。

2. 其它常用函数

在 CUDA 编程中，除了核函数外，还有一些常用的辅助函数，用于内存管理、数据传输和同步操作。这些函数是高效利用 GPU 计算资源的重要工具。以下是一些最常用的 CUDA 函数及其简要介绍。

`cudaMalloc` 函数用于在设备（GPU）上分配内存。它的原型为 `cudaError_t cudaMalloc(void **devPtr, size_t size)`。`devPtr` 是一个指向指针的指针，用于存储分配的设备内存地址，`size` 是要分配的内存大小（以字节为单位）。例如，以下代码在设备上分配一个浮点型数组：

```
1 float *d_A;
2 size_t size = N * sizeof(float);
3 cudaError_t err = cudaMalloc((void **)&d_A, size);
4 if (err != cudaSuccess) {
5     // 处理错误
6 }
```

设备内存分配示例

`cudaFree` 函数用于释放之前使用 `cudaMalloc` 分配的设备内存。其原型为 `cudaError_t cudaFree(void *devPtr)`，`devPtr` 是指向要释放的设备内存的指针。

`cudaMemcpy` 函数用于在主机（Host）和设备（Device）之间传输数据。它的原型为 `cudaError_t cudaMemcpy(void *dst, const void *src, size_t count, cudaMemcpyKind kind)`。`dst` 和 `src` 分别是目标和源内存地址，`count` 是传输的数据大小（以字节为单位），`kind` 指定传输的方向（如 `cudaMemcpyHostToDevice` 或 `cudaMemcpyDeviceToHost`）。例如，以下代码将数据从主机传输到设备：

```
1 float *h_A = (float *)malloc(size);
2 cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
```

数据传输示例

在并行计算中，同步操作至关重要。CUDA 提供了多个同步函数，用于控制核函数的执行顺序和数据一致性。cudaDeviceSynchronize 函数用于在主机代码中同步设备操作，确保之前启动的所有核函数在继续执行主机代码之前完成。其原型为 `cudaError_t cudaDeviceSynchronize(void)`。

`__syncthreads` 是一个内置的线程块级别的同步函数，用于在线程块内同步所有线程。每个线程在继续执行之前，都会等待所有线程都到达同步点。这个函数在核函数内部使用。

3. 编程约束

在 CUDA 编程中，有一些特定的约束和限制，这些限制由 CUDA 架构的硬件特性决定，理解这些限制有助于编写高效且稳定的 CUDA 程序。

首先，线程块的大小有严格的限制。每个线程块中的线程数不能超过 1024 个，且线程块可以是一维、二维或三维的。一维线程块的最大尺寸为 1024，二维线程块的每个维度最大尺寸为 1024，而三维线程块的每个维度最大尺寸为 64。这意味着一个二维线程块可以是 32x32（总计 1024 个线程），而一个三维线程块可以是 8x8x16（总计 1024 个线程）。

网格的大小也有具体限制。一维网格的最大尺寸为 $2^{31} - 1$ ，二维网格的每个维度最大尺寸为 65535，三维网格的每个维度最大尺寸同样为 65535。这些限制确保了在不同维度配置网格时的灵活性和可扩展性。

在硬件资源方面，每个 CUDA 设备有固定数量的寄存器、共享内存和常量内存。每个线程最多可使用 255 个 32 位寄存器，每个流式多处理器（SM）内的 32 位寄存器总数为 64K。共享内存是线程块内所有线程共享的高速内存，每个 SM 可用的共享内存总量通常为 64KB 到 96KB，但在最新的架构（如 Hopper 架构）中，这个值可以达到 164KB 甚至 228KB。常量内存的总大小通常为 64KB，用于存储在核函数执行过程中不变的数据。

每个 CUDA 设备还有一些其他的限制。例如，warp 的大小是固定的，每个 warp 包含 32 个线程。每个 SM 上同时可驻留的最大线程块数和 warp 数有限。例如，一个 SM 上最多可以驻留 32 个线程块和 64 个 warp，总线程数限制为 2048。每个设备能够同时运行的并发核函数（resident grids）数量也有限，不同设备的具体值可以在官网上的[CUDA C 程序设计指导](#)中的表格 21 中查阅。

4.CUDA 程序基本流程

CUDA 程序的基本流程可以通过主机（CPU）和设备（GPU）之间的协同工作来实现，主要包括数据传输、核函数调用和结果返回等步骤。整个流程可以分为四个主要步骤，如图4所示。

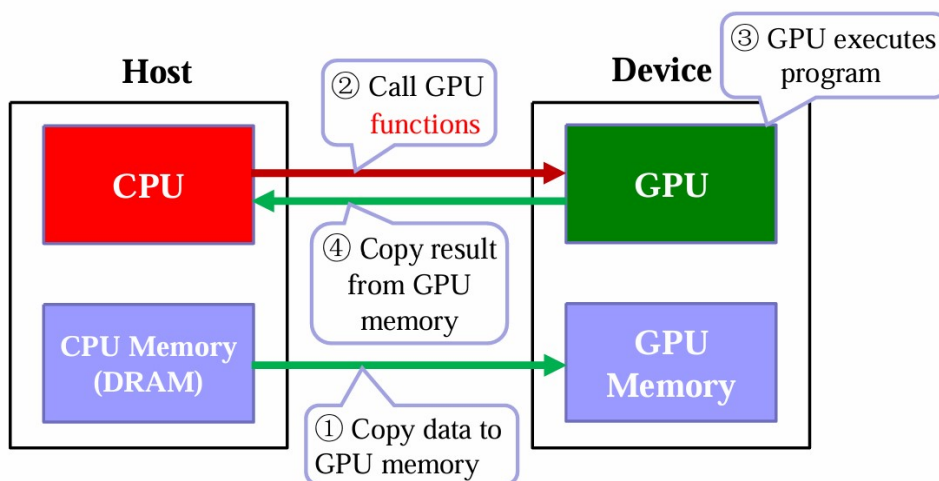


图 4 CUDA 程序基本流程

首先，程序在主机端开始，将需要处理的数据从主机内存复制到设备内存。这一步是通过调用 `cudaMemcpy` 函数来实现的，将数据从 CPU 内存传输到 GPU 内存。这一步确保了 GPU 能够访问和处理所需的数据，从而准备好进行并行计算。

接下来，主机端调用核函数，启动 GPU 的并行计算。核函数通过特殊的语法（如 `<<blocks, threads>>`）配置网格和线程块的维度，以确定并行计算的规模。核函数在设备端执行，利用 GPU 的计算能力对数据进行处理。此时，GPU 中的每个线程根据其索引处理不同的数据部分，实现高效的并行计算。

第三步，GPU 执行核函数中的并行计算任务。GPU 的各个线程协同工作，处理分配给它们的数据片段。核函数内部使用线程索引（如 `blockIdx`、`blockDim` 和 `threadIdx`）来确定每个线程处理的数据位置，确保每个线程都有独立的工作负载。在核函数执行期间，主机端通常会等待 GPU 完成所有计算任务，可以通过同步函数（如 `cudaDeviceSynchronize`）确保计算任务的正确完成。

最后一步，计算完成后，将结果从设备内存复制回主机内存。这一步再次使用 `cudaMemcpy` 函数，将处理结果从 GPU 内存传输回 CPU 内存。主机端接收到结果后，可以对其进行进一步的处理、验证或存储。这个数据传输过程确保了计算结果能够返回到主机，从而完成整个 CUDA 计算任务。

5. 主要优化方法

在 CUDA 编程中，优化程序性能是充分利用 GPU 强大计算能力的关键。通过合理使用各种优化技术，可以显著提高 CUDA 程序的执行效率和性能。以下是一些常见的优化方法。

首先，合理使用共享内存是优化 CUDA 程序性能的重要手段之一。共享内存是每个线程块内的所有线程共享的高速内存，其访问延迟比全局内存低得多。因此，在需要频繁访问和修改的数据处理中，使用共享内存可以显著提高性能。例如，可以将需要频繁访问的数据从全局内存加载到共享内存中，线程间进行数据交换时直接使用共享内存，从而减少全局内存访问次数。

其次，使用常量内存优化只读数据访问。常量内存是专门用于存储在核函数执行过程中不会改变的数据，其访问延迟低且具有较高的带宽。如果核函数中的大部分线程都需要访问相同的只读数据，可以将这些数据存储在常量内存中，利用常量内存的缓存机制提高数据访问效率。

此外，确保全局内存访问的共线性也是一种有效的优化策略。在 CUDA 中，内存访问模式对性能有很大影响。全局内存访问的共线性是指连续的线程访问连续的内存地址，这种访问模式能够充分利用内存带宽，减少内存访问延迟。在编写核函数时，尽量保证线程按照共线性的方式访问内存，可以显著提高内存访问效率。

另一个优化方法是使用适当的线程块和网格配置。不同的 CUDA 设备具有不同的硬件资源限制，如每个 SM (Streaming Multiprocessor) 中的最大线程数、寄存器数量和共享内存大小。通过实验和性能分析，找到最优的线程块大小和网格配置，可以最大限度地利用 GPU 资源，提高计算效率。例如，对于需要大量计算的核函数，可以使用较大的线程块，以充分利用 SM 中的并行计算资源；对于内存访问密集的核函数，可以使用较小的线程块，以提高内存访问效率。

最后，减少分支和分支发散也是提高 CUDA 程序性能的重要方法。分支发散是指同一 warp 中的不同线程执行不同路径的情况，会导致性能下降。在编写核函数时，尽量减少条件语句和分支操作，或者将条件判断放在 warp 级别而不是线程级别，可以减少分支发散，提高执行效率。

二、实验环境

(一) 实验环境简介

本次实验在我的个人计算机上完成，CPU 为 AMD Ryzen 9 5900HX，8 核 16 线程，基准速度 3.30GHz，双通道内存共 32GB，速度 3200MHz，GPU 为 Ampere 架构的 NVIDIA GeForce RTX 3080 Laptop，16GB 显存，具有 48 组流式多处理器，每组 128 个共 6144 个 CUDA 核心。在[官网](#)上能够查询到我 GPU 的 CUDA Capability 为 8.6，图5中展示了使用 GPU-Z 查看的更为详细的 GPU 硬件信息。

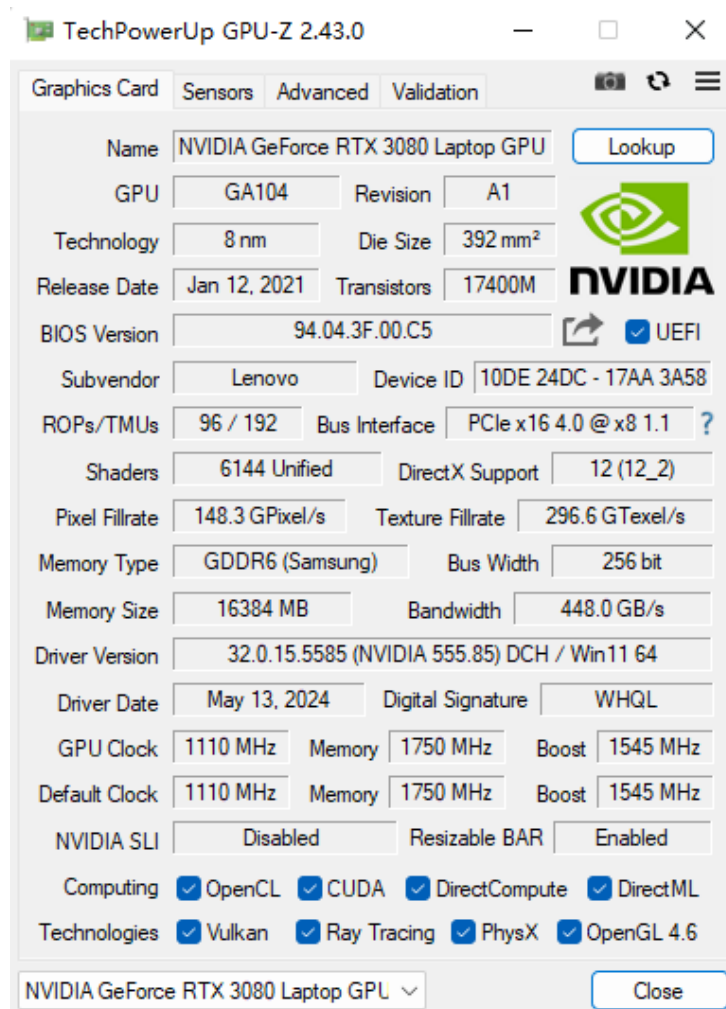


图 5 GPU-Z 查询到的 GPU 硬件信息

此外，一些软件环境的版本见下表。

表 1 软件版本表

软件名	版本
Windows	10.0.22000.2538
WSL	2.1.5.0
WSL Linux 内核	5.15.146.1-2
Ubuntu	22.04.3 LTS
NVIDIA-SMI & Driver	555.42.03
Cuda compilation tools	V12.3.52
g++	11.4.0

（二）环境配置流程

为了完成本次作业，我需要为我所使用的个人计算机配置 CUDA 环境。斯坦福校方为本次作业准备了云服务器虚拟机，并提供了对应的配置脚本以及配置教程，以在其所提供的虚拟机中快速的配置 CUDA 环境。但我作为北航的学生，没有访问其所提供的虚拟机的权限，因而它的脚本和配置教程仅仅能作为参考，我需要结合我的具体运行环境进行配置。当然，或许还有使用 docker 的配置方式，但是我并没有使用这种方案，而是直接在我的 WSL 中进行配置。

环境配置和之后完成实验，均基于[作业仓库](#)的提交 fd0f705，这也是我完成作业时它最新的版本。在这个版本提供的配置脚本将会安装 12.3 版本的 CUDA，和 545.23.06 版本的 NVIDIA-SMI 和 Driver，这意味着我至少应该安装在此之上的版本。我查看我本地现有的版本，发现我 NVIDIA-SMI 和 Driver 的版本为 512，其最高支持的 CUDA 版本为 11.6。

由于我的系统版本与斯坦福提供的虚拟机一致，也就是说符合配置脚本中的需求，同为 x86-64 下的 Ubuntu 22.04，因此我首先尝试直接运行配置脚本尝试环境的配置。第一次运行仓库中的环境配置脚本 install.sh 失败了，再次运行后安装脚本成功执行。执行完毕后，我通过 nvcc 查看到已经成功安装了 12.3 版本的 CUDA，但是虽然安装了 545 版本的 NVIDIA-SMI，但是 Driver 的版本仍为 512，无法支持 12.3 版本的 CUDA。

为此，我在 WSL 中卸载了我的 GPU 驱动程序，并回到宿主机，尝试使用宿主机中的 GeForce Experience 软件更新 GPU 驱动，但是以失败告终，在升级驱动时会报错。于是我去官网下载最新版本的驱动，并手动安装，成功。此时我进入 WSL 中，发现其中的 NVIDIA-SMI 和 Driver 已经同步得到了更新，版本均为 555，能够运行 12.3 版本的 nvcc 编译产生的 CUDA 程序。

三、翻译

实验要求以及云服务器配置文档的翻译在“简单的 CUDA 渲染器”以及“AWS 设置说明”两文件中，均提供了 Markdown 版本和 PDF 版本。

四、实验一：SAXPY

该部分是作业中的第一个实验，用于帮助学生熟悉 CUDA 编程模型。这部分的任务是使用 CUDA 重新实现实验一中的 SAXPY 函数。

（一）实验背景

SAXPY 函数是一种用于线性代数的基本操作，特别是在并行计算和数值计算中。它的主要目的是对向量进行缩放和相加。具体来说，它的操作是对两个向量 X 和 Y 进行元素级的操作，将向量 X 的每一个元素乘以常量 a ，然后加上向量 Y 中对应的元素，得到结果。

（二）实验代码浅析

该部分的核心代码主要位于两个文件中，分别是主程序 `mian.cpp` 和 `saxpy.cu`。

1. 主程序

这个主程序的功能是测试和运行一个基于 CUDA 的 SAXPY 算法实现。程序首先设置默认的数组大小为 100,000,000 元素，并通过解析命令行参数允许用户指定数组大小。然后，程序初始化三个数组 `xarray`、`yarray` 和 `resultarray`，并将 `xarray` 和 `yarray` 的每个元素设置为其索引值对 10 取余的结果，而 `resultarray` 的每个元素初始为 0。

在初始化完成后，程序调用 `printCudaInfo` 函数来打印 CUDA 设备的信息，然后运行三次 `saxpyCuda` 函数来进行实际的 SAXPY 计算。每次计算中，`saxpyCuda` 函数使用 CUDA 在 GPU 上执行 SAXPY 操作，将结果存储在 `resultarray` 中。最后，程序释放之前分配的数组内存并结束执行。

2. 算法实现

`saxpy.cu` 文件定义并实现了在 GPU 上运行的 SAXPY 算法。核心的 CUDA 内核函数 `saxpy_kernel` 被标记为 `__global__`，它在 GPU 上运行，并计算给定向量的 SAXPY 操作。每个线程根据其索引计算结果，并将其存储在结果数组中。

`void saxpyCuda(int N, float alpha, float* xarray, float* yarray, float* resultarray)` 函数在 CPU 上运行，负责管理内存和调用 CUDA 内核，这也是在这个实验中需要实现的

函数。首先，它计算需要处理的总字节数，并确定每个块中的线程数和所需的块数。然后，它在 GPU 上分配内存，并将输入数组从 CPU 复制到 GPU。接着，它启动 CUDA 内核 saxpy_kernel，在 GPU 上并行执行计算。计算完成后，结果被复制回 CPU，并记录和打印执行时间及带宽。最后，释放在 GPU 上分配的内存。

printCudaInfo 函数打印当前机器上可用的 CUDA 设备信息，包括设备数量、名称、多处理器数量、全局内存大小和 CUDA 版本等详细信息。这个函数有助于了解执行环境中的 GPU 规格。

（三）实验过程

该实验比较基础和简单，只需按照前文“CUDA 程序基本流程”一节中所叙述的，完成设备内存的分配，将数据从主机拷贝到设备，调用核函数，最后再将结果从设备拷贝回主机即可。但是由于不太熟练，在这个简单的过程中一开始也犯了一些错误。

首先，在申请内存和进行数据拷贝时，一开始我忘了将使用的内存大小乘以 sizeof(float)，导致了申请的内存长度不够，发生了内存的越界访问问题，使得程序在运行的过程中，会产生代码为 700 的错误，错误消息 an illegal memory access was encountered。在看到这个错误之后，我进行排查，找到了问题所在，并进行了修改。

此外，在核函数执行完成后，我本应使用 cudaDeviceSynchronize() 进行同步，这导致我计算的执行时间惊人的快，几乎为 0。正如实验说明中的叙述，由于主机和设备上的程序异步执行，如果我不进行同步，则此时只计算了 API 调用的时间，而没有计算核函数执行的时间。

（四）结果与结论

在 saxpy 目录下完成本部分实验后，使用 make 进行编译，然后使用 ./cudaSaxpy 进行执行，得到图6所示结果。

```
akashi@LIU-Yizhou-Legion-w11:~/develop/parallel/cuda/stanford-cs149-asst3/saxpy (master)$ ./cudaSaxpy
-----
Found 1 CUDA devices
Device 0: NVIDIA GeForce RTX 3080 Laptop GPU
  SMs:      48
  Global mem: 16384 MB
  CUDA Cap:  8.6
-----
Running 3 timing tests:
Execution time: 0.006 ms
Effective BW by CUDA saxpy: 219.909 ms      [5.082 GB/s]
Execution time: 0.003 ms
Effective BW by CUDA saxpy: 219.366 ms      [5.095 GB/s]
Execution time: 0.003 ms
Effective BW by CUDA saxpy: 213.409 ms      [5.237 GB/s]
```

图 6 SAXPY 实验程序执行结果

通过图中展示的时间可以看出，GPU 执行核函数的时间非常短，绝大多数时间都花费在了数据的传输上。

接下来回答实验说明中提出的问题。第一个问题是与基于顺序 CPU 的 SAXPY 实现的性能对比，由于我们没有完成这个部分，因此没有办法进行对比，但可以想象出，将会有很大的性能提升。

第二个问题第一部分是解释由两组计时器提供的结果之间的差异。通过图中展示的时间可以看出，GPU 执行核函数的时间非常短，绝大多数时间都花费在了数据的传输上。这说明 GPU 高度并行使得计算时间大幅缩短，但受限于内存、总线等设备的限制，在数据传输上需要消耗大量的时间。并且在实际的工作过程中，实际的数据传输速率受到各种因素影响，往往与理论上限仍有差距。

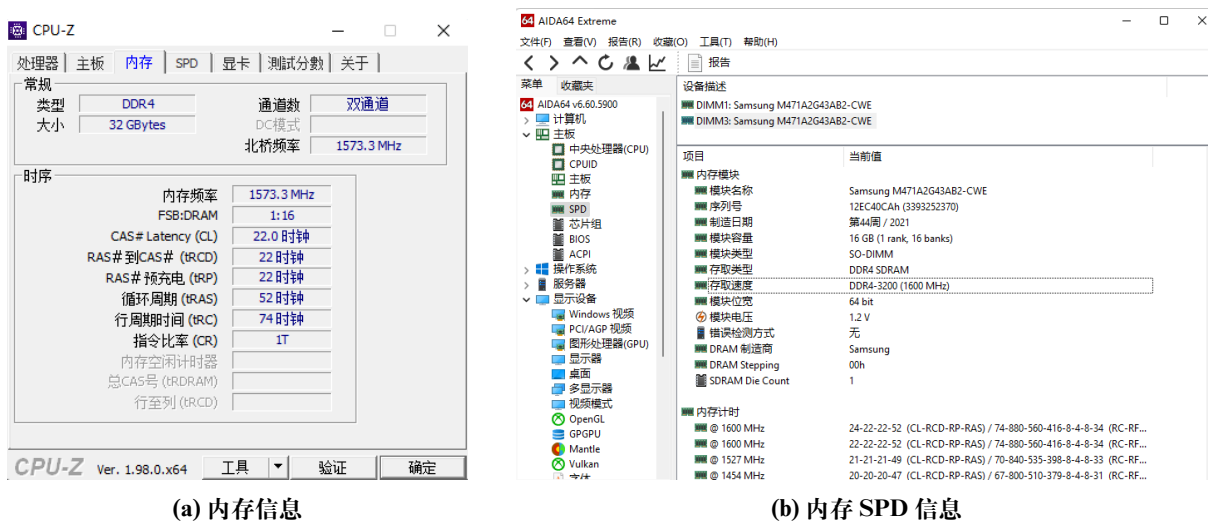


图 7 内存信息

我使用相关软件查看了我自己设备的一些信息，图7a中，我使用 CPU-Z 查看内存相关信息，可以看到我的两根内存条正确的组成了双通道，并且正常工作在 1600MHz，由于 DDR4 内存在上升沿和下降沿均进行数据读写，所以数据读写频率 3200MHz。然后我使用 AIDA64 软件查看内存 SPD 中的信息，可以看到模块位宽为 64 位。据此，我可以计算我内存的理论带宽上限：

$$3200MHz \times 64bit \times 2 = 51200MB/s$$

之后，我发现在 AIDA64 中也可以直接查询到主板内存总线的带宽，如图8所示，与上面的计算结果相符。此外，从图5中可以发现，GPU 的带宽为 448.0GB/s，显然内存总线的带宽较低，可能成为瓶颈。不过实验一程序运行过程中，实际的传输速度比这个值还要小很多。

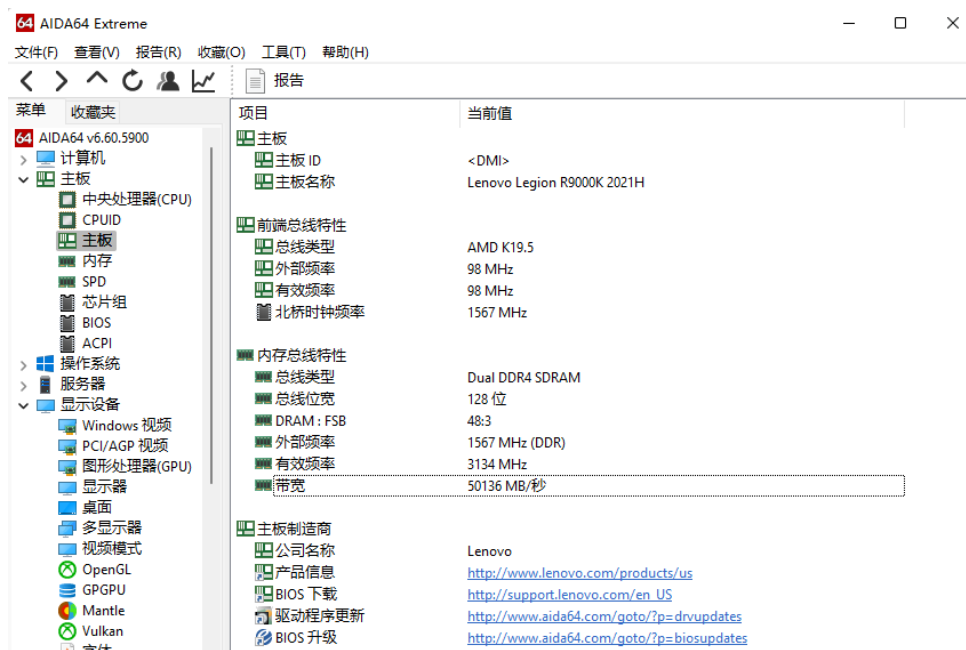


图 8 主板信息

五、实验二：并行前缀和

该部分是作业中的第二个实验，同样用于帮助学生熟悉 CUDA 编程模型。实验一主要涉及到主机对于 GPU 核函数的调用，而实验二在实验一的基础上，还需要同学们自行进行串行算法设计，并编写核函数。虽然该实验名称为并行前缀和，但实际上该实验有两个部分，其中一个为并行前缀和，而另一部分是调用第一部分实现的并行前缀和，进一步实现查找数组中重复项的下标。

（一）实验背景

并行前缀和算法是一种在并行计算中常用的技术，特别是在需要高效处理大数据集的场景中。它可以用来快速计算数组元素的累积和，以及在许多其他并行算法中作为基础步骤。查找数组中连续重复项的下标是一个典型的问题，可以通过并行前缀和算法来高效解决。

（二）实验代码浅析

该实验核心代码均需要学生完成，因此只对代码文件做简单的介绍，不做过多赘述。

主程序 `main.cpp` 主要用于调用并行前缀和或查找重复项的算法实现，以完成指定的任务。任务由命令行参数指定，可以指定的内容包括但不限于：选择执行并行前缀和任务或者是查找重复项任务；选择不同的输入类型，默认为随机。`scan.cu` 文件主要包

括并行前缀和以及查找重复项的算法实现，这两部分都是需要学生完成的。

(三) 实验过程

1. 算法设计

独占前缀和的算法实现参照实验说明中和课件中提供的算法，如图9所示。该算法的伪代码也在实验说明中给出。该算法分为上扫和下扫两个阶段。在上扫阶段中，算法分组的收集元素的和，将其保存在固定的位置。具体来说，算法将每组中间的元素与最后的元素求和，并保存在组内最后的位置。组的长度从2开始，逐渐增加到不能覆盖完整数组的最大长度，也就是说，下一组参与求和的两个元素，恰好是上一组中保存两个和的两个元素。在下扫阶段，算法先将最后一个元素置零，然后将收集到的和分配到各个元素中。

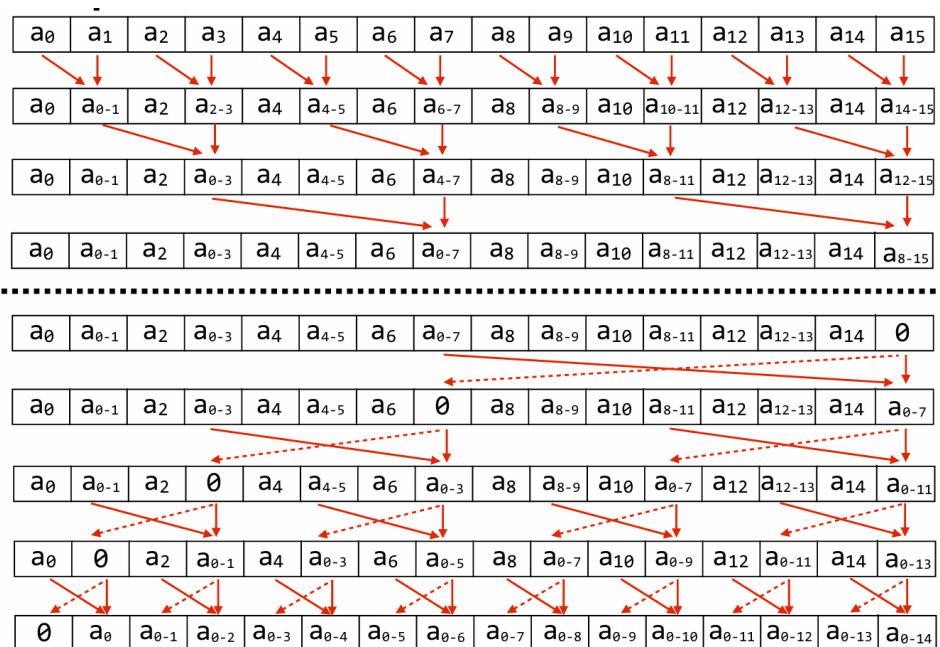


图9 并行独占和算法示意图

查找重复项的算法需要用到独占前缀和的计算，主要用后者实现对于下标的收集。具体的算法如图10所示。具体来说，算法首先使用一个指示器，标记原始数组的某一元素是否与下一元素相同。然后对该指示器计算独占前缀和，用于表示前面已有重复项的数量，为最后一步并行的结果收集做指导。最后，检查独占前缀和中发生变化的项，其代表的就是这里有需要收集的下标，然后根据其内容，也就是表示前面已有的数量，将其放置在结果数组的对应位置中即可。总数可以使用最后一个元素的值来表示。

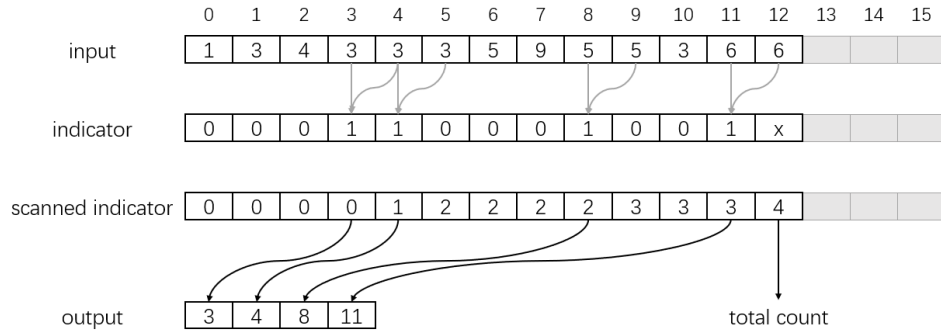


图 10 查找重复项算法示意图

2. 实现过程

该算法实现难度较低，依照算法与伪代码逐步实现即可。但在实现过程中，我也遇到了一些小问题，以下是实现独占前缀和的两个核函数，分别表示上扫阶段和下扫阶段。

```

1 __global__ void upsweep_kernel(const int two_d, const int two_dplus1, int * input,
2   int * output, const int length) {
3   int idx = (blockIdx.x * blockDim.x + threadIdx.x) * two_dplus1;
4   if (idx < length) {
5       output[idx + two_dplus1 - 1] = input[idx + two_dplus1 - 1] + input[idx +
6       two_d - 1];
7   }
8 }
9
10 __global__ void downsweep_kernel(const int two_d, const int two_dplus1, int * input
11   , int * output, const int length) {
12   int idx = (blockIdx.x * blockDim.x + threadIdx.x) * two_dplus1;
13   if (idx < length) {
14       // input maybe same with output
15       int temp = input[idx + two_dplus1 - 1];
16       output[idx + two_dplus1 - 1] = input[idx + two_dplus1 - 1] + input[idx +
17       two_d - 1];
18       output[idx + two_d - 1] = temp;
19   }
20 }

```

独占前缀和核函数

其中变量的命名与题目中的伪代码一致。其中, `two_dplus1` 表示组的长度, 而 `two_d` 表示组长度的一半, 用于查找组中间的元素。在一开始编写核函数时, 我忽略了 `input` 和 `output` 可能指向同一个地方, 而在使用时我传入了相同的参数, 导致了问题。

（四）结果与结论

独占前缀和的实验结果如图11所示。可以看到性能超越了满分的要求。

Scan Score Table:			
Element Count	Ref Time	Student Time	Score
1000000	6.330	1.458	1.25
10000000	8.833	5.031	1.25
20000000	13.998	8.648	1.25
40000000	27.287	15.806	1.25
Total score:			5/5

图 11 独占前缀和结果

查找重复项的实验结果如图12所示。性能基本达到了满分的要求，只是在第一个小数据集上的性能没有达标。

Find_repeats Score Table:			
Element Count	Ref Time	Student Time	Score
1000000	1.928	2.510	0.960159362549801
10000000	12.943	11.415	1.25
20000000	23.993	20.388	1.25
40000000	46.163	36.956	1.25
Total score:			4.7101593625498/5

图 12 查找重复项结果

据我分析，可能的原因主要有以下两个方面。一方面，我的算法实现局部性较差，对 cache 的利用率较为低下；同时在 strided 访存模式下，一次访问 16 字节数据，但仅使用 4 字节，同样降低了访存效率，增加了访存次数。另一方面，该题本意可能是希望学生使用共享内存进行优化，这将带来显著的性能提升，尤其是在较小的数据集上。具体的原因可以使用 NVIDIA Nsight Compute 等工具进行进一步的分析。

六、实验三：简单圆形渲染器

该实验是本次作业最主要的内容，设计并实现一个 CUDA 并行圆形渲染器的渲染部分。

（一）实验代码浅析

1. 代码概述

该实验代码较多，分别位于许多不同的文件，包括但不限于以下文件。

首先是头文件。例如，`circleRenderer.h` 声明抽象的 `CircleRenderer` 类，声明了一系列的虚函数，用于设置、动画、渲染等操作。`cudaRenderer.h` 声明使用 CUDA 进行渲染的 `CudaRenderer` 类，`refRenderer.h` 声明 CPU 渲染的 `RefRenderer` 类，它们都是 `CircleRenderer` 的派生类。`cycleTimer.h` 声明用于高精度计时的 `CycleTimer` 类，主要用于性能测量；`noise.h` 声明噪声生成函数；`ppm.h` 声明用于写入 PPM 图像文件的函数；`platformgl.h` 根据平台包含相应的 GLUT 库头文件，用于 OpenGL 编程；`sceneLoader.h` 声明加载场景的函数；`util.h` 声明一些实用宏和函数，例如颜色夹具函数 `CLAMP`。

接着是一些 `cpp` 源文件。`main.cpp` 是程序的主入口点，处理命令行参数，初始化渲染器，可以选择使用不同的渲染方式，处理不同的渲染任务。当主程序使用渲染器时，首先调用 `allocOutputImage` 分配图像内存，然后使用 `loadScene` 来加载和初始化场景，接着调用 `setup` 进行初始化，最后由 `CheckBenchmark` 或其他函数，对每一帧图像调用 `clearImage`、`advanceAnimation` 和 `render` 来更新和渲染图像。`display.cpp` 包含 OpenGL 渲染相关的代码，用于显示图像；`noise.cpp` 实现噪声生成逻辑；`ppm.cpp` 实现将图像数据写入 PPM 文件的逻辑；`sceneLoader.cpp` 实现加载不同场景的逻辑。实验需要重点关注的也包括 `refRenderer.cpp`，它实现 `RefRenderer` 类，是 CPU 渲染的参考实现。

然后是一些 CUDA 源文件。`exclusiveScan.cu_inl` 文件实现了一种用于 CUDA 中的独占前缀和操作的內联函数；`benchmark.cu` 包含用于性能测试的 CUDA 代码。`lookupColor.cu` 文件定义了一个用于在 CUDA 内核中查找颜色的设备內联函数，这个函数通过一个给定的坐标值在预定义的颜色映射表中进行插值，以获取对应的颜色值。`noise-Cuda.cu_inl` 文件提供了一个在 CUDA 内核中生成噪声的设备函数。最后是最为核心的 `cudaRenderer.cu`，这是一个不正确的并行 CUDA 实现，需要由学生修改完成。这个错误的实现不满足原子性和顺序要求，这点将在之后详细介绍。

此外，还有用于指导编译的 `Makefile` 以及用于测试的 `checker.py` 等文件，在实验过程中不需要过多关注。

2. 渲染过程浅析

对于不同的场景，渲染器有着不同的帧更新逻辑，用于在渲染新的帧之前对元素进行调整。

而对于渲染过程，在串行实现中，任何场景的每个元素都会在 `render` 中被限定在

一个矩形范围内，对于该范围内的任何一个像素，会被调用 `shadePixel` 进行真正像素的叠加。在这个函数里，位于矩形范围内，但在真正需要渲染的圆形范围外的像素会被忽略。对于雪花相关的场景，其像素颜色需要通过 `lookupColor` 进行颜色插值来获取，而其余场景则不需要，只需按照实验说明中的叙述，进行带有透明度的像素叠加即可。

在提供的错误 CUDA 实现中，渲染器将不同的圆圈分配给不同的线程，在每个线程中，通过两层循环来调用 `shadePixel`，渲染各个圆形所在的矩形框中的像素。而此外，没有确保圆形渲染顺序的同步操作，或是对于某一像素的互斥访问，这也是导致这个实现不具备原子性和顺序性的原因。

（二）实验过程与结果

在该实验中，我尝试了多个不同版本的实现。

在第一个版本中，我尝试使用像素级的并行，每个线程负责一个像素，按顺序处理所有的圆形。以下为这个基础版本的并行渲染核函数。

```

1  __global__ void kernelRenderCircles() {
2      short imageWidth = cuConstRendererParams.imageWidth;
3      short imageHeight = cuConstRendererParams.imageHeight;
4      short pixelX = blockIdx.x * blockDim.x + threadIdx.x;
5      short pixelY = blockIdx.y * blockDim.y + threadIdx.y;
6
7      // [0, imageWidth - 1], [0, imageHeight - 1]
8      if (pixelX >= imageWidth || pixelY >= imageHeight) {
9          return;
10     }
11
12     int circlesCnt = cuConstRendererParams.numCircles;
13     // inverse coordinate
14     float invWidth = 1.f / imageWidth;
15     float invHeight = 1.f / imageHeight;
16     float2 pixelCenterNorm = make_float2(invWidth * (static_cast<float>(pixelX) +
17                                           invHeight * (static_cast<float>(pixelY) +
18                                           .5f));
19
20     float4 * imgPtr = (float4 *) (&cuConstRendererParams.imageData[(pixelY *
21 imageWidth + pixelX) << 2]);
22
23     for (int circleIdx = 0; circleIdx < circlesCnt; circleIdx++) {
24         float3 circlePos = *(float3*)&cuConstRendererParams.position[circleIdx *
25 3]);
26         shadePixel(circleIdx, pixelCenterNorm, circlePos, imgPtr);
27     }
28 }

```

```

23     }
24 }

```

基础并行渲染核函数

在这个版本中，对于每个像素，同样也是每个线程，我计算了它在图像中的位置，计算像素中心点，然后在循环中调用 `shadePixel` 进行像素的更新。该版本的计时结果如图13所示，可以看到效率较低，与要求仍有较大差距。

Score table:			
Scene Name	Ref Time (T_ref)	Your Time (T)	Score
rgb	0.3193	0.2246	12
rand10k	1.9441	27.1578	2
rand100k	17.3583	286.5537	2
pattern	0.4095	3.8454	4
snowsingle	11.3327	266.7253	2
biglittle	10.2616	33.1404	6
Total score:			28/72

图 13 渲染器结果 1

此时我注意到在 `shadePixel` 函数中有一个分支，对于每个像素的每个圆圈都会在 GPU 上进行执行，消耗时间较长，因此我决定尝试将这个分支移动到 CPU 上。在这个过程进行的同时，我拆开了 `shadePixel` 函数，将其逻辑直接写在圆形渲染函数中，并为两个不同的场景提供不同的圆圈渲染核函数，由 CPU 在调用渲染函数时就固定好分支，避免在 GPU 上执行分支操作，同时省略掉不必要的计算，并将本地使用局部变量保存各个像素的状态，减少对全局内存的访问。但当我完成一半，也就是完成对于通常场景的修改后，我发现这并没有什么作用，于是我放弃了我的修改。结合之后的实验，现在想来使用局部变量进行优化应该是有用的，但是对于分支的优化可能带来了意想不到的影响。后来我了解到了 C++ 的 `constexpr`，我使用它进行了优化，但出乎我意料的是，时间不仅没有缩短，某些场景下反而还显著增加了。经过我的测试，`constexpr` 确实按照我的预期，将分支提前到了 CPU 上，且分支的次数也极大幅度的减小了，从每帧每像素每圆圈，减少到了每帧一次。所以执行时间的提升让我感到很不理解。图14展示了该情况下的耗时。

Score table:			
Scene Name	Ref Time (T_ref)	Your Time (T)	Score
rgb	0.3348	0.2716	12
rand10k	1.9896	67.7241	2
rand100k	17.648	658.5634	2
pattern	0.348	9.0189	2
snowsingle	11.0788	255.9593	2
biglittle	10.2962	66.3691	4
Total score:			24/72

图 14 渲染器结果 2

随后我开始思考如何优化算法，以获取更高的效率。我上网搜寻思路，意识到可以将像素分块处理，在每个块内仅处理与该块相交的圆圈，并将相交的圆圈信息保存在该块的共享内存中，以提升访存效率。在该算法中，每个线程块中的每个线程，不仅对应到具体的每个像素，还并行的查找相交的圆圈。具体来说，有一个每次处理块中线程数量圆的循环。在循环中首先，块中的各个线程根据当前块的范围，调用提供的函数 `circleInBoxConservative` 粗略的计算与自己对应的圆圈是否与该块相交，相交则标记指示器为 1。然后与上一个实验类似，计算指示器的独占和，然后找到这一次循环中与该块相交的圆保存在共享内存中。接着各个线程代表各个像素，依次访问相交的圆，调用 `shadePixel` 更新像素。在这整个过程中，`shadePixel` 更新的一直是本地的像素状态。当所有圆圈处理完成后，将其写回全局内存。该算法及其实现能够获得该实验的满分，结果见图15。

Score table:			
Scene Name	Ref Time (T_ref)	Your Time (T)	Score
rgb	0.2818	0.2892	12
rand10k	2.1399	1.7569	12
rand100k	19.2139	15.3417	12
pattern	0.3966	0.3761	12
snowsingle	12.3252	5.4545	12
biglittle	11.2808	12.2781	12
Total score:			72/72

图 15 渲染器结果 3

最后，我略微的进行最后的优化，调整了一些细节，进一步减少了对全局内存的访问，结果如图16所示，效果不太显著。

Score table:			
Scene Name	Ref Time (T_ref)	Your Time (T)	Score
rgb	0.2647	0.2634	12
rand10k	2.1289	1.7474	12
rand100k	19.2398	15.3243	12
pattern	0.367	0.3543	12
snowsingle	12.3295	5.3821	12
biglittle	11.3413	12.1646	12
		Total score:	72/72

图 16 渲染器结果 4

此外，如果将 **Position** 的三个值和半径合在一起，不仅能进一步减少访存，还能帮助 16B 对齐，进一步提高访存效率，但我没有实现这点。

参考文献

- [1]NVIDIA A100 Tensor Core GPU Architecture. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>
- [2]Nvidia Hopper Architecture New Features. Available: <https://zhuanlan.zhihu.com/p/676437405>
- [3]Work-efficient parallel exclusive scan ($O(N)$ work). Available: https://gfxcourses.stanford.edu/cs149/fall23/lecture/dataparallel/slide_16
- [4]CMU 15418 Spring 2023 Assignment 2: A Simple CUDA Renderer. Available: <https://zhuanlan.zhihu.com/p/683223356>